Text & URL Summarizer — Documentation

Ruchitha Chintanaboina | Flask + Hugging Face Transformers (Google Colab)

1. Project overview

This project is an NLP-based text summarizer. It accepts either raw pasted text or a news article URL, and returns a short, condensed summary while preserving the key meaning. It runs as a small Flask web app, developed and tested inside a Google Colab notebook and exposed to the internet using ngrok.

2. Tech stack

- **Flask** — lightweight Python web framework that serves the page and handles form submissions.
- **Hugging Face Transformers** — provides the pretrained summarization model.
- **facebook/bart-large-cnn** — the specific model used; a BART model fine-tuned for abstractive summarization on news-style text.
- **newspaper3k** — extracts clean article text from a given URL (strips ads, navigation, etc.).
- **pyngrok** — creates a temporary public URL so the Colab-hosted app can be opened in a browser outside Colab.

3. How it works, step by step

1. Required libraries are installed: flask, pyngrok, transformers, torch, newspaper3k, and lxml.
2. Three folders are created: templates/ (for the HTML page), static/ (for CSS), and uploads/ (reserved for future use, e.g. file uploads).
3. app.py is written to disk. It loads the summarization pipeline once at startup, then defines a single route ("/") that handles both displaying the form (GET) and processing a submission (POST).
4. The summarize_text_or_url() function checks whether a URL or raw text was submitted. If a URL is given, newspaper3k downloads and parses the article to extract its main text. That text (or the pasted text) is then passed to the summarizer.
5. The summarizer pipeline generates a summary capped between 50 and 150 tokens, using greedy decoding (do_sample=False) for consistent, repeatable output.
6. templates/index.html is written — a simple form with a textarea for pasted text and an input field for a URL, plus a result card that shows the generated summary.
7. static/style.css is written — a dark gradient theme with frosted-glass "card" panels for the form and result.
8. Any previously running Flask or ngrok processes are killed to avoid port conflicts, then app.py is launched in the background and its logs are written to flask.log.
9. pyngrok opens a public tunnel to local port 8000 and prints a public URL — this is the link used to open the app in any browser.

4. How to run this notebook yourself

1. Open the notebook in Google Colab (a free Google account is enough — no local setup needed).
2. Run the first cell to install all dependencies (this can take a couple of minutes the first time, since it also downloads PyTorch).

3. Run the cells in order — each %%writefile cell creates one of the project's files (app.py, index.html, style.css) inside the Colab environment.
4. Get a free ngrok auth token from ngrok.com (sign up, then copy the token from your dashboard) and replace the token string in the last cell with your own — tokens should never be shared publicly once a notebook becomes public.
5. Run the final cells to start the Flask server and open the ngrok tunnel. The printed Public URL is your live, shareable summarizer.
6. Open that URL, paste text or a news article link, and click Summarize.

5. Notes and tips

- ngrok free-tier URLs are temporary — they expire when the Colab session ends or disconnects, so this setup is best for demos, not permanent hosting.
- The summarization model runs faster with a Colab GPU runtime enabled (Runtime → Change runtime type → GPU); on CPU it still works but is slower.
- Very long articles may need to be truncated before summarization, since BART has a maximum input length.
- For a permanent, always-on version, the same Flask app can be deployed to a free host like Render or Railway instead of relying on Colab + ngrok.

6. Key file reference

```
app.py — Flask routes and the summarize_text_or_url() function
templates/index.html — the form and summary display page
static/style.css — dark gradient theme styling
```